

DP Computer Science: B 2.3.3 Looping

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain the **purpose and application** of loops in programming
 - Differentiate between different loop types (**for**, **while**, **do-while** in Java; **for** and **while** in Python)
 - Explain how **conditional statements and operators** control loop execution
 - Implement various loop types to solve different programming problems
-

Key Vocabulary

Term	Definition
Loop	A programming construct that repeats a block of code multiple times
Iteration	A single execution of the loop body
for loop	A loop that iterates a specific number of times (count-controlled)
while loop	A loop that continues while a condition is true (condition-controlled)
do-while loop	(Java only) Executes loop body at least once before checking condition
break	A statement that exits a loop immediately
continue	A statement that skips the rest of the current iteration
Infinite loop	A loop that never terminates because its condition is always true

Term	Definition
Nested loop	A loop inside another loop

IB ATL Skills

Skill	Description
Thinking	Critical thinking (analysing and evaluating issues); creative thinking (generating novel ideas)
Communication	Exchanging thoughts and information effectively through interaction
Self-Management	Managing time and tasks effectively
Research	Finding, interpreting, judging, and creating information

2. Introduction to Loops

What are Loops?

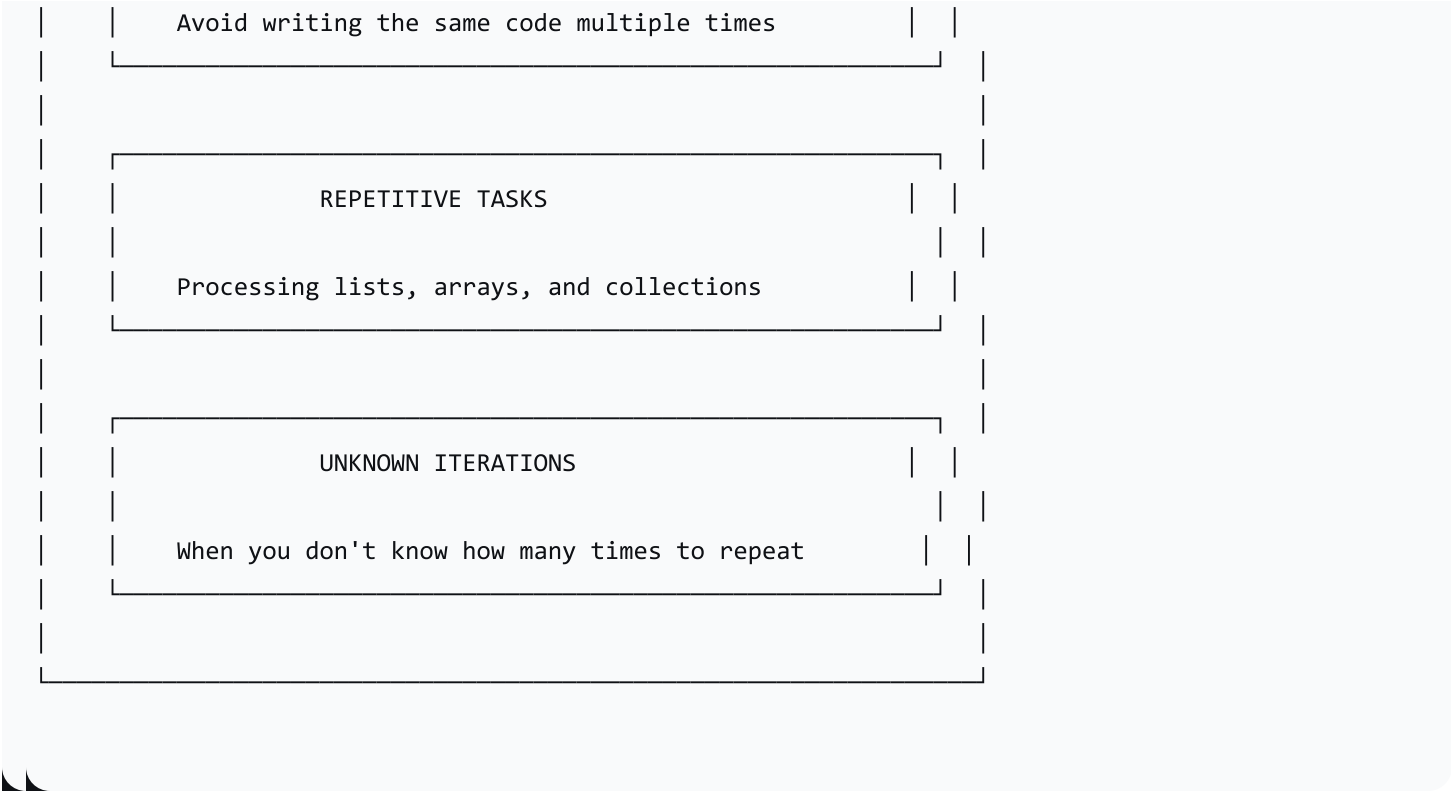
Loops allow you to repeat a block of code multiple times. They are essential for performing repetitive tasks efficiently.

Why Use Loops?

text

WHY USE LOOPS?

EFFICIENCY



Real-World Examples

Scenario	Loop Type
Processing a list of students	for loop
Reading user input until valid	while loop
Displaying menu repeatedly	while loop
Calculating factorial	for loop

3. The for Loop

What is a for Loop?

A `for` loop is used when you know the number of iterations in advance (count-controlled loop).

for Loop Syntax

python

```
# Python
for variable in range(start, end, step):
    # Code to repeat
```

java

```
// Java
for (initialisation; condition; update) {
    // Code to repeat
}
```

for Loop Examples

Example 1: Counting 1 to 5

python

```
# Python
for i in range(1, 6):
    print(i)
# Output: 1, 2, 3, 4, 5
```

java

```
// Java
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
// Output: 1, 2, 3, 4, 5
```

Example 2: Sum of Numbers

python

```
# Python
total = 0
for i in range(1, 11):
```

```
total += i
print(f"Sum: {total}") # Output: Sum: 55
```

java

```
// Java
int total = 0;
for (int i = 1; i <= 10; i++) {
    total += i;
}
System.out.println("Sum: " + total); // Output: Sum: 55
```

Example 3: Iterating Over a List

python

```
# Python
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# Output: apple, banana, cherry
```

java

```
// Java
String[] fruits = {"apple", "banana", "cherry"};
for (String fruit : fruits) {
    System.out.println(fruit);
}
// Output: apple, banana, cherry
```

range() Function (Python)

python

```
# range(stop) - from 0 to stop-1
for i in range(5):
    print(i) # 0, 1, 2, 3, 4
```

```
# range(start, stop) - from start to stop-1
for i in range(2, 6):
    print(i) # 2, 3, 4, 5

# range(start, stop, step) - with a step
for i in range(0, 10, 2):
    print(i) # 0, 2, 4, 6, 8

# range(start, stop, step) - decreasing
for i in range(5, 0, -1):
    print(i) # 5, 4, 3, 2, 1
```

4. The while Loop

What is a while Loop?

A `while` loop is used when the number of iterations is not known in advance (condition-controlled loop).

while Loop Syntax

python

```
# Python
while condition:
    # Code to repeat
```

java

```
// Java
while (condition) {
    // Code to repeat
}
```

while Loop Examples

Example 1: Counting 1 to 5

python

```
# Python
i = 1
while i <= 5:
    print(i)
    i += 1
# Output: 1, 2, 3, 4, 5
```

java

```
// Java
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
// Output: 1, 2, 3, 4, 5
```

Example 2: User Input Validation

python

```
# Python
num = -1
while num < 0 or num > 100:
    num = int(input("Enter a number between 0 and 100: "))
print(f"Valid number: {num}")
```

java

```
// Java
import java.util.Scanner;
Scanner scanner = new Scanner(System.in);
int num = -1;
while (num < 0 || num > 100) {
    System.out.print("Enter a number between 0 and 100: ");
    num = scanner.nextInt();
}
```

```
}  
System.out.println("Valid number: " + num);
```

Example 3: Sum of Digits

python

```
# Python  
num = 123  
sum_digits = 0  
while num > 0:  
    sum_digits += num % 10  
    num //= 10  
print(f"Sum of digits: {sum_digits}") # Output: 6
```

java

```
// Java  
int num = 123;  
int sumDigits = 0;  
while (num > 0) {  
    sumDigits += num % 10;  
    num /= 10;  
}  
System.out.println("Sum of digits: " + sumDigits); // Output: 6
```

5. do-while Loop (Java Only)

What is a do-while Loop?

A `do-while` loop executes the loop body at least once before checking the condition.

java

```
// Java  
do {
```

```
// Code to repeat
} while (condition);
```

do-while Example

```
java

// Java: Menu example
import java.util.Scanner;
Scanner scanner = new Scanner(System.in);
int choice;

do {
    System.out.println("Menu:");
    System.out.println("1. Option 1");
    System.out.println("2. Option 2");
    System.out.println("3. Exit");
    System.out.print("Enter choice: ");
    choice = scanner.nextInt();
} while (choice != 3);
```

Comparison: while vs. do-while

Feature	while	do-while
First Execution	Condition checked first	Body executed first
Minimum Executions	0 (if condition false)	1 (always)
Use Case	Unknown iterations	At least one iteration required

6. Loop Control Statements

break Statement

break exits a loop immediately.

python

```
# Python
for i in range(1, 11):
    if i == 5:
        break
    print(i)
# Output: 1, 2, 3, 4
```

java

```
// Java
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
// Output: 1, 2, 3, 4
```

continue Statement

continue skips the rest of the current iteration and moves to the next one.

python

```
# Python
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# Output: 1, 2, 4, 5
```

java

```
// Java
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
```

```
        continue;
    }
    System.out.println(i);
}
// Output: 1, 2, 4, 5
```

Infinite Loops

python

```
# Python: Infinite Loop (missing increment)
i = 0
while i < 10:
    print(i)
    # Missing: i += 1
```

java

```
// Java: Infinite Loop (condition always true)
while (true) {
    System.out.println("This runs forever!");
}
```

7. Nested Loops

What are Nested Loops?

Nested loops are loops inside other loops. The inner loop completes all its iterations for each iteration of the outer loop.

Nested Loop Example

python

```
# Python: Multiplication table
for i in range(1, 4):
```

```
for j in range(1, 4):
    print(f"{i} x {j} = {i * j}")
print() # Blank line after each row
```

java

```
// Java: Multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i + " x " + j + " = " + (i * j) + " ");
    }
    System.out.println();
}
```

Visualising Nested Loops

text

NESTED LOOPS VISUALISATION	
Outer loop (i = 1):	
Inner loop (j = 1, 2, 3):	
1 x 1 = 1, 1 x 2 = 2, 1 x 3 = 3	
Outer loop (i = 2):	
Inner loop (j = 1, 2, 3):	
2 x 1 = 2, 2 x 2 = 4, 2 x 3 = 6	
Outer loop (i = 3):	
Inner loop (j = 1, 2, 3):	
3 x 1 = 3, 3 x 2 = 6, 3 x 3 = 9	

Common Nested Loop Patterns

python

```
# Pattern 1: Star pattern
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()

# Output:
# *
# **
# ***
# ****
# *****
```

python

```
# Pattern 2: Number pattern
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end="")
    print()

# Output:
# 1
# 12
# 123
# 1234
# 12345
```

8. Worksheet Activities

Activity 1: Sum of Even Numbers

python

```
# Write a program that calculates the sum of even numbers from 1 to 100
# Hint: Use a for loop and an if statement
```

Activity 2: Factorial Calculator

python

```
# Write a program that calculates the factorial of a given number
# Example: 5! = 5 × 4 × 3 × 2 × 1 = 120
```

Activity 3: Number Guessing Game

python

```
# Write a program that:
# 1. Generates a random number between 1 and 100
# 2. Asks the user to guess the number
# 3. Provides feedback (too high, too low, correct)
# 4. Continues until the user guesses correctly
# Hint: Use a while loop and the random module
```

Activity 4: Multiplication Table

python

```
# Write a program that prints a multiplication table
# for numbers 1 to 10 using nested loops
```

Activity 5: Prime Number Checker

python

```
# Write a program that checks if a given number is prime
# Hint: Use a for loop to check divisors from 2 to sqrt(n)
```

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	What are loops? Why use loops?
for Loop	15 min	Structure, examples; Activity 1
while Loop	15 min	Structure, examples; Activity 2
Loop Control/Nested	10 min	break, continue, nested loops
Conclusion	10 min	Key takeaways; further exploration

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of loops
Coding Challenges	Assign small coding projects using loops

11. Differentiation

Level	Strategy
Support	Provide scaffolded examples; use visual aids
Challenge	Explore more complex nested loops; algorithm challenges

12. Resources

- **Presentation** (Slide Deck)
- **eBook** (B2.3.3 with examples)
- **Worksheets:** Loop exercises
- **Worksheet Answers**

13. Summary: Key Takeaways

Loop Type	When to Use	Key Feature
for	Known number of iterations	Count-controlled
while	Unknown number of iterations	Condition-controlled
do-while (Java)	At least one execution	Post-test loop
break	Exit loop immediately	Loop control
continue	Skip current iteration	Loop control
nested	Loop inside loop	Complex repetition

text

KEY REMINDERS

✓

Loops repeat code efficiently

✓

for loop = known iterations

✓

while loop = condition-controlled

✓

break exits a loop

✓

continue skips current iteration

✓

Nested loops = loop inside a loop

✓

Avoid infinite loops

"Loops are the workhorses of programming—they handle repetitive tasks efficiently and are essential for processing data."